Reviewer Pack — Minimal Local Ring Norm Correlation with Communication Compl…

Entry 930de1c43d1a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 09:46:57 UTC

Minimal Local Ring Norm Correlation with Communication Complexity Growth

Entry ID: `930de1c43d1a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 09:46:57 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every instance φ of size n, the minimal norm of a local ring extension associated with φ is linearly correlated with the growth rate of the communication complexity of φ, such that MinimalNorm(R) = Θ(GrowthRate(φ)).

Rationale (proposer's reasoning):

> Local rings provide a framework for studying algebraic structures in computational models. Their norms could capture inherent complexities in computation processes like communication complexity, potentially revealing new structural insights.

Taxonomy category: `LocalRingTheory × CommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `de14079db8f6f04c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all instances φ with size n ≤ 40, the Pearson correlation coefficient between MinimalNorm(R) and GrowthRate(φ) across 30 seeds is greater than or equal to 0.8, and the upper bound of the correlation coefficient is within Θ(n^2). The conjecture is falsified if any seed produces a correlation coefficient less than 0.5, or an upper bound outside of Θ(n^2).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Ring Norm AND Communication Complexity - Local Ring Theory intersection Communication Complexity - norm correlation in local rings related to communication complexity growth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        # Generate a random instance φ of size n
        return [random.randint(1, 100) for _ in range(n)]

    def compute_local_ring_norm(instance):
        # Compute the minimal norm of the local ring extension associated with instance
        norm = sum(x**2 for x in instance)
        return Fraction(norm).limit_denominator()

    def compute_growth_rate(instance):
        # Compute the growth rate of the communication complexity of instance
        n = len(instance)
        return (n * (n + 1)) / 2

    instances_tested = 0
    total_norm = 0
    total_growth_rate = 0
    n_max = 0

    for _ in range(30):
        n = random.randint(5, 40)
        instance = generate_instance(n)
        norm = compute_local_ring_norm(instance)
        growth_rate = compute_growth_rate(instance)

        if norm == 0 or growth_rate == 0:
```

```
            continue

        total_norm += norm
        total_growth_rate += growth_rate
        instances_tested += 1
        n_max = max(n_max, n)

    if instances_tested < 30:
        return {
            "metric_name": "Correlation",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "Insufficient instances tested"
        }

    mean_norm = Fraction(total_norm).limit_denominator() / instances_tested
    mean_growth_rate = total_growth_rate / instances_tested

    # Calculate Pearson correlation coefficient
    numerator = sum((x - mean_norm) * (y - mean_growth_rate) for x, y in zip([compute_local_ring_r
    denominator = math.sqrt(sum((x - mean_norm)**2 for x in [compute_local_ring_norm(generate_inst
    correlation_coefficient = numerator / denominator if denominator != 0 else 0

    return {
        "metric_name": "Correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.8 and correlation_coefficient <= (n_max**
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
lds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.823968934169614, 'instances_tested': 30, 'n_
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.0505121462659857, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.7737180492060466, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.8892994128727052, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.9238224847281539, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.0138733505103734, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.093735226055394, 'instances_tested': 30, 'n_
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.7354275603423222, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.9568708229093922, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.0772826655604686, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.0277372410223555, 'instances_tested': 30, 'n
TRIAL: {'metric_name': 'Correlation', 'metric_value': 1.0122769346308382, 'instances_tested': 30, 'n
RESULT: FALSIFIED counterexample="" first_failing_seed=23
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code computes the minimal norm of a local ring extension as the sum of squares of elements, which is not correct for local rings in general. The growth rate of communication complexity is computed using a formula that does not correspond to any known measure of communication complexity. These failures indicate a metric definition bug.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results show that for at least one seed (first_failing_seed=23), the Pearson correlation coefficient between MinimalNorm(R) and GrowthRate(φ) | next: Re-evaluate the metrics used in the test to ensure they correctly represent minimal norm of a local ring extension and the growth rate of communication complexity. If the metrics are correct, further investigate why the correlation coefficient failed to meet the threshold for one or more

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20785
2	preregistration	ollama_remote	glm4:latest	0	0	13545
3	novelty	ollama_remote	glm4:latest	0	0	17192
4	novelty	ollama_remote	glm4:latest	0	0	11506
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13661
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19381
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18466

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14846
9	critic	ollama_remote	glm4:latest	0	0	18091
10	judge	ollama_remote	glm4:latest	0	0	16688

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 164162 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/930de1c43d1a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/930de1c43d1a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/930de1c43d1a.tar.gz` (if generated)